



Конспект по теме «Введение в SQL»

Содержание:

1. Введение
2. DDL — определение структуры данных
3. DML — работа с данными
4. DCL — управление правами доступа
5. Пошаговое руководство по управлению доступом
6. TCL — управление транзакциями
7. Итог

1. Введение

SQL (Structured Query Language) — язык запросов для работы с реляционными базами данных. Он позволяет создавать структуры данных, управлять ими, работать с данными, контролировать доступ и управлять транзакциями.

SQL делится на четыре основные группы команд:

- **DDL (Data Definition Language)** — определение структуры данных
- **DML (Data Manipulation Language)** — работа с данными
- **DCL (Data Control Language)** — управление правами доступа
- **TCL (Transaction Control Language)** — управление транзакциями

2. DDL — определение структуры данных

Команды DDL отвечают за создание, изменение и удаление объектов базы данных (таблиц, баз, индексов, представлений).

CREATE — создание базы данных, таблицы, представления или индекса.

Примеры:

sql

```
CREATE DATABASE dvd_rental;

CREATE TABLE customer (

    id SERIAL PRIMARY KEY,

    fio CHAR(50) UNIQUE,

    dob DATE

);

CREATE VIEW underage AS

    SELECT fio FROM customer WHERE TIMESTAMPDIFF(YEAR, dob, CURDATE()) < 18;

CREATE UNIQUE INDEX dob_idx ON customer(dob);
```

ALTER — изменение структуры объектов (переименование, добавление/изменение столбцов)

Примеры:

sql

```
ALTER TABLE customer RENAME TO persons;

ALTER TABLE persons ADD password CHAR(20) NOT NULL;

ALTER TABLE persons MODIFY fio VARCHAR(100);
```

DROP — удаление объектов базы данных

Примеры:

sql

```
DROP DATABASE dvd_rental;

DROP TABLE persons;

DROP INDEX dob_idx ON persons;

DROP VIEW underage;
```

Важно: Не используйте DROP на продакшн-системах без резервных копий!

3. DML — работа с данными

Команды DML позволяют добавлять, изменять, удалять и получать данные из таблиц.

SELECT — выборка данных

Примеры:

sql

```
SELECT * FROM persons;           -- все записи

SELECT fio, dob FROM persons;    -- определённые столбцы

SELECT fio, dob FROM persons WHERE id > 25; -- с условием

SELECT COUNT(*) FROM persons;    -- количество записей
```

INSERT — добавление новых данных

Примеры:

sql

```
INSERT INTO persons (fio) VALUES ('Петров Пётр');

INSERT INTO persons VALUES (6, 'Иванов Иван', '1985-07-12');

INSERT INTO persons (fio) SELECT fio FROM customers;
```

UPDATE — изменение существующих данных

Примеры:

sql

```
UPDATE persons SET fio = 'Максим Сергеевич' WHERE id = 1;

-- Важно: без WHERE обновятся все записи!
```

DELETE — удаление данных

Примеры:

sql

```
DELETE FROM persons WHERE id IN (1, 3, 7);
```

```
DELETE FROM persons; -- удалит все записи (использовать осторожно!)
```

4. DCL — управление правами доступа

Команды DCL отвечают за выдачу и отзыв прав пользователей.

GRANT — предоставление прав

Примеры:

sql

```
CREATE USER 'sys_temp'@'localhost' IDENTIFIED BY 'password';
```

```
GRANT SELECT, INSERT ON sakila.* TO 'sys_temp'@'localhost';
```

```
GRANT ALL PRIVILEGES ON *.* TO 'sys_temp'@'localhost' WITH GRANT OPTION;
```

REVOKE — отзыв прав

Примеры:

sql

```
REVOKE INSERT, DELETE ON sakila.* FROM 'sys_temp'@'localhost';
```

```
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'sys_temp'@'localhost';
```

```
DROP USER 'sys_temp'@'localhost';
```

DENY — запрет прав (отсутствует в MySQL, есть в других СУБД)

Совет: для безопасности не давайте пользователям больше прав, чем нужно. Например, аналитикам достаточно прав только на SELECT.

5. Пошаговое руководство по управлению доступом

На практике работа с правами доступа сводится к нескольким ключевым шагам: создание пользователя, выдача ему прав и, при необходимости, их отзыв.

Рассмотрим этот процесс подробно на примере.

Шаг 1: Создание пользователя

Всё начинается с создания новой учётной записи. Представим, что нам нужен временный пользователь для выполнения системных задач или для интеграции с другим сервисом. Назовём его `sys_temp`.

Синтаксис:

sql

```
CREATE USER 'имя_пользователя'@'хост' IDENTIFIED BY 'пароль';
```

- **'имя_пользователя'**: Уникальное имя для новой учётной записи.
- **'хост'**: Адрес, с которого разрешено подключение. `localhost` означает, что подключиться можно только с той же машины, где работает СУБД. Использование `'%'` позволяет подключаться с любого сетевого адреса.
- **'пароль'**: Пароль для аутентификации пользователя.

Пример:

sql

```
CREATE USER 'sys_temp'@'localhost' IDENTIFIED BY 'password';
```

Эта команда создаст пользователя `sys_temp`, который сможет подключаться к серверу баз данных только локально, используя указанный пароль.

Шаг 2: Просмотр списка пользователей

Чтобы убедиться, что пользователь успешно создан, или просто проверить список существующих учётных записей, можно выполнить запрос к системной таблице `mysql.user`.

Пример:

sql

```
SELECT user, host FROM mysql.user;
```

Выполнение этого запроса покажет таблицу со списком всех пользователей и хостов, с которых им разрешён доступ. В этом списке вы должны увидеть `sys_temp`.

Шаг 3: Предоставление прав (GRANT)

Сразу после создания пользователь не имеет никаких прав — он не может даже просматривать список баз данных. Чтобы дать ему необходимые разрешения, используется команда `GRANT`.

Синтаксис:

`GRANT [права] ON [объект] TO 'пользователь'@'хост';`

- **[права]:** Тип разрешения. Например, `SELECT` (чтение), `INSERT` (вставка), `UPDATE` (обновление) или `ALL PRIVILEGES` (все возможные права).
- **[объект]:** Уровень доступа. `sakila.*` означает «все таблицы в базе данных `sakila`», а `*.*` — «все таблицы во всех базах данных».

Пример (выдача полных прав):

sql

```
GRANT ALL PRIVILEGES ON *.* TO 'sys_temp'@'localhost';
```

Этот запрос предоставляет пользователю sys_temp максимальные полномочия на всём сервере. На практике такие широкие права выдаются только административным учётным записям.

Шаг 4: Просмотр выданных прав

В любой момент можно проверить, какими правами обладает тот или иной пользователь. Для этого существует удобная команда SHOW GRANTS.

Пример:

sql

```
SHOW GRANTS FOR 'sys_temp'@'localhost';
```

Результат выполнения покажет список всех разрешений, назначенных пользователю sys_temp.

Шаг 5: Отзыв прав (REVOKE)

В целях безопасности часто требуется не только выдавать права, но и ограничивать их. Для этого существует команда REVOKE, которая является обратной к GRANT.

Синтаксис:

```
REVOKE [права] ON [объект] FROM 'пользователь'@'хост';
```

Пример (ограничение прав):

Допустим, мы хотим, чтобы sys_temp мог только читать данные из базы sakila, но не изменять их. Для этого нужно отозвать у него права на INSERT, UPDATE и DELETE.

sql

```
REVOKE INSERT, UPDATE, DELETE ON sakila.* FROM 'sys_temp'@'localhost';
```

После выполнения этой команды пользователь сохранит право SELECT, но уже не сможет модифицировать данные.

Шаг 6: Удаление пользователя

Когда учётная запись больше не нужна, её следует удалить, чтобы избежать потенциальных уязвимостей. Для этого используется команда DROP USER.

Пример:

sql

```
DROP USER 'sys_temp'@'localhost';
```

Эта команда полностью и безвозвратно удалит пользователя из системы.

Ключевой принцип безопасности:

Всегда следуйте принципу наименьших привилегий. Предоставляйте пользователям только тот набор прав, который минимально необходим им для выполнения своих задач. Например, аналитической системе достаточно прав только на SELECT. Важно помнить, что синтаксис управления пользователями может незначительно отличаться в разных СУБД, например, в PostgreSQL, но фундаментальные принципы (GRANT, REVOKE) остаются неизменными.

6. TCL — управление транзакциями

Транзакции позволяют выполнять группу операций как единое целое — либо всё успешно, либо ничего.

- **START TRANSACTION (BEGIN)** — начало транзакции
- **COMMIT** — подтверждение изменений
- **ROLLBACK** — откат изменений
- **SAVEPOINT** — установка точки сохранения внутри транзакции
- **ROLLBACK TO SAVEPOINT** — откат к точке сохранения

Пример транзакции в MySQL:

```
START TRANSACTION;
```

```
UPDATE accounts SET balance = balance - 1000 WHERE id = 10;
```

```
SAVEPOINT sp1;
```

```
UPDATE accounts SET balance = balance + 1000 WHERE id = 27;
```


-- Если ошибка, откат к sp1

ROLLBACK TO sp1;

UPDATE accounts **SET** balance = balance + 1000 **WHERE** id = 37;

COMMIT;

7. Итог

- При создании базы данных и таблиц используйте DDL-команды (CREATE, ALTER). Для удаления Баз данных или таблиц используйте DROP.
- Для работы с данными — DML (SELECT, INSERT, UPDATE, DELETE).
- Для управления пользователями и правами — DCL (CREATE USER, GRANT, REVOKE, DROP USER).
- Для контроля целостности операций — TCL (START TRANSACTION, COMMIT, ROLLBACK).
- Обязательно используйте точку с запятой «;» для разделения команд.
- Пишите ключевые слова SQL заглавными буквами для удобства чтения.
- Внимательно проверяйте права пользователей, чтобы не допустить лишних разрешений.
- Работайте с транзакциями аккуратно, особенно при изменении нескольких связанных таблиц.